QuarkyLab GPU Cluster

Researcher Guide

Remote access + the commands to run your research

Researchers get a normal interactive shell with direct GPU access — no SLURM required. This guide assumes you are comfortable at the command line.
Admin, key requests, and help: ***contact Kyle on our shared Discord channel.***

1 Access (one-time)

You reach QuarkyLab through a **Cloudflare Tunnel** — no VPN, no inbound ports, and the server's IP stays hidden. Login is by **SSH key**. Do this once.

1.1 Create an SSH key and send the public half

```
ssh-keygen -t ed25519 -C "you@lab"
cat ~/.ssh/id_ed25519.pub
```

Send that `.pub` line to Kyle on our shared Discord channel. He adds it to your account and tells you your **username** (e.g. `cephandrius`). **Never** send the private key (`id_ed25519`, the one without `.pub`).

1.2 Install cloudflared

macOS

`brew install cloudflared`

Windows (PowerShell)

`winget install --id Cloudflare.cloudflared`

Linux

Package or static binary from `https://github.com/cloudflare/cloudflared/releases/latest`

1.3 Route SSH through the tunnel

Add this to `~/.ssh/config` (create the file if it does not exist):

```
Host quarkylab
    HostName quarkylab.kylemason.org
    ProxyCommand cloudflared access ssh --hostname %h
    IdentityFile ~/.ssh/id_ed25519
```

1.4 Connect

```
ssh <your-username>@quarkylab
```

`cloudflared` dials out on demand and your key logs you in (accept the host-key prompt with `yes` the first time). You land on a normal shell in your home directory.

> **Note**
>
> An email sign-in gate (Cloudflare Access) may be added later; if so, your first connection will also open a one-time browser login. Nothing you set up above changes.

2 The machine

- 1 × **Quadro RTX 8000, 48 GB** — driver 550.163.01, CUDA 12.1 stack.
- `nvidia-smi` shows GPU status and current usage. You share this **one physical GPU** with students' batch jobs and Fernanda — glance at it before grabbing a lot of VRAM.

3 GPU and the ML environment

The curated stack is an Apptainer image whose `ml` conda environment has **Python 3.11, PyTorch 2.5.1+cu121, TensorFlow 2.21**, plus NumPy, pandas, SciPy, scikit-learn, and the HuggingFace stack. Image: `/data/containers/base.s` (fast local copy at `/workspace/containers/base.sif`).

```
# run a script on the GPU
apptainer exec --nv /data/containers/base.sif python train.py

# interactive shell inside the container
apptainer shell --nv /data/containers/base.sif

# quick sanity check
apptainer exec --nv /data/containers/base.sif \
  python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name
    (0))"
# -> True Quadro RTX 8000
```

- `python` inside the image is already the `ml` env (`/opt/conda/envs/ml`).
- `--nv` is what exposes the GPU — omit it and CUDA is invisible.
- Your **home** and **current directory** are auto-mounted; bind extra paths with `-B /host/path:/container/path`.

4 Your own environment (optional)

No sudo, but your home is yours. For a custom stack, install Miniforge in your home:

```
curl -L -o ~/miniforge.sh https://github.com/conda-forge/miniforge/releases/latest/
    download/Miniforge3-Linux-x86_64.sh
bash ~/miniforge.sh -b -p ~/miniforge3
~/miniforge3/bin/conda create -n myenv python=3.11 pytorch-cuda=12.1 ...
```

The NVIDIA driver is already on the host — install a CUDA-matched PyTorch/TF in your env, or just use the container.

5 Storage

Path	Use	Notes
`~` = `/workspace/researchers/<you>`	code + results	150 GB quota, backed up nightly
`/workspace/scratch/<you>`	fast scratch	not backed up; files older than 14 days auto-purged
`/data/shared`	shared datasets / docs	read-only
`/data`	bulk (NFS, ~23 TB)	ask Kyle for a writable project dir if you need one

6 Moving data

`scp` and `rsync` use your SSH config, so they route through the tunnel with no extra flags:

```
scp bigfile.tar quarkylab:~/                 # upload
scp quarkylab:~/results.csv .                # download
rsync -avP ./dataset/ quarkylab:~/dataset/   # sync a folder
```

7 Long-running work: use tmux

SSH sessions can drop (laptop sleep, network blips). A detached session keeps your job alive:

```
tmux new -s run        # start; launch your training inside
# Ctrl-b then d to detach; reconnect later:
ssh quarkylab
tmux attach -t run
```

(`screen` works too; `nohup cmd &` for fire-and-forget.)

8 Etiquette

- One physical GPU, shared. Running directly (outside SLURM) you are **not** VRAM-capped — check `nvidia-smi` first and don't OOM others.
- For multi-day campaigns there is a `research` SLURM partition (interactive allowed, priority over students) if you'd rather queue and preempt-share — optional, coordinate with Kyle.

Help: contact Kyle on our shared Discord channel.